

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO1 Statement:	<i>Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)</i>		
Student Name:	J KARTHIK	Reg. No.:	192472136
Section / Batch:	CSE – AI / 2024	Date:	

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

Question Type	Focus Area	Questions	Marks
Application-Based Questions	Apply Master Theorem and asymptotic analysis to real-world scenarios	Q1 – Q2	Q1 –50 Q2 –50
GRAND TOTAL:		100 Marks	

SECTION A – APPLICATION BASED QUESTIONS

Code of Conduct

I, **J KARTHIK / 192472136** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: J KARTHIK_____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)	Total Marks
Q1 (50 Marks)						
Q2 (50 Marks)						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Question 1: Online Video Streaming Platform

Application

A video streaming platform processes user data using the recurrence $T(n) = 2T(n/2) + n$. The recurrence models a divide-and-conquer strategy where the data is split into two equal halves, each processed recursively, with linear-time work (n) done to combine/merge the results at every level (e.g., merging processed chunks of streaming metadata).

Algorithm Used

Master Theorem for Divide-and-Conquer Recurrences

The Master Theorem solves recurrences of the standard form $T(n) = aT(n/b) + f(n)$, where:

- a = number of subproblems the problem is divided into
- n/b = size of each subproblem
- $f(n)$ = cost of work done outside the recursive calls (dividing + combining)

Algorithm Analysis (Step-by-Step Solution)

Recurrence	$T(n) = 2T(n/2) + n$
a (subproblems)	2
b (division factor)	2
f(n)	n
$n^{\log_b a}$	$n^{\log_2 2} = n^1 = n$

Comparing $f(n)$ with $n^{\log_b a}$: $f(n) = n$ and $n^{\log_b a} = n$, so $f(n) = \Theta(n^{\log_b a})$.

This satisfies Case 2 of the Master Theorem, which applies when $f(n)$ grows at the same rate as $n^{\log_b a}$.

Result: $T(n) = \Theta(n \log n)$

Interpretation & Scalability

The $\Theta(n \log n)$ complexity means the streaming platform's data-processing cost grows only slightly faster than linearly as the number of users (n) increases. This is a highly scalable growth rate — commonly seen in efficient divide-and-conquer algorithms such as merge sort — so the platform can comfortably handle large and growing volumes of user data without a drastic drop in performance.

Question 2: Social Media Feed Processing

Application

A social media platform processes posts using the recurrence $T(n) = T(n-1) + n$. This models a linear recursive algorithm where, at each step, the problem size is reduced by only one unit (e.g., processing one additional post at a time), and n units of work are performed at each recursive call (such as re-ranking or scoring against all previously processed posts).

Algorithm Used

Substitution Method for Solving Recurrences

The substitution method solves a recurrence by repeatedly expanding (unrolling) the recursive term until a base case is reached, then summing the accumulated work.

Algorithm Analysis (Step-by-Step Expansion)

$$T(n) = T(n-1) + n$$

$$T(n-1) = T(n-2) + (n-1) \implies T(n) = T(n-2) + (n-1) + n$$

$$T(n-2) = T(n-3) + (n-2) \implies T(n) = T(n-3) + (n-2) + (n-1) + n$$

Continuing this pattern down to the base case $T(0) = c$:

$$T(n) = T(0) + 1 + 2 + 3 + \dots + (n-1) + n$$

$$T(n) = c + n(n+1)/2$$

Result: $T(n) = \Theta(n^2)$

Interpretation & Scalability

The $\Theta(n^2)$ growth rate is quadratic — performance degrades rapidly as the number of posts (n) increases. Doubling the input roughly quadruples the processing time. This indicates the algorithm is poorly scalable for a large, fast-growing social media feed and should ideally be replaced with a divide-and-conquer or incremental-update approach for large-scale deployment.

Question 6: Banking Transaction Processing System

Application

A banking system uses a divide-and-conquer algorithm with recurrence $T(n) = 4T(n/2) + n^2$ to process transaction records. The problem (transaction batch) is split into 4 subproblems of half the size, and n^2 work is performed at each level (e.g., cross-verifying transaction pairs during fraud/consistency checks).

Algorithm Used

Master Theorem for Divide-and-Conquer Recurrences

Algorithm Analysis (Step-by-Step Solution)

Recurrence	$T(n) = 4T(n/2) + n^2$
a (subproblems)	4
b (division factor)	2
f(n)	n^2
$n^{\log_b a}$	$n^{\log_2 4} = n^2$

Comparing $f(n)$ with $n^{\log_b a}$: $f(n) = n^2$ and $n^{\log_b a} = n^2$, so $f(n) = \Theta(n^{\log_b a})$.

This satisfies Case 2 of the Master Theorem.

Result: $T(n) = \Theta(n^2 \log n)$

Interpretation & Scalability

The $\Theta(n^2 \log n)$ complexity is significantly worse than linear or $n \log n$ growth. For large transaction volumes, processing time increases sharply — much faster than the growth of the input itself. This suggests the banking system's algorithm may face performance bottlenecks at scale and could benefit from optimization (e.g., reducing the branching factor a or the per-level work $f(n)$) to remain efficient as transaction volumes grow.

Question 7: Search Engine Index Builder

Application

A search engine builds its index using the recurrence $T(n) = T(n/2) + 1$. This models an algorithm that repeatedly halves the problem size (e.g., a binary-search-style lookup while building index structures) and performs only constant ($O(1)$) work at each recursive step.

Algorithm Used

Substitution Method for Solving Recurrences

Algorithm Analysis (Step-by-Step Expansion)

$$T(n) = T(n/2) + 1$$

$$T(n/2) = T(n/4) + 1 \implies T(n) = T(n/4) + 1 + 1 = T(n/4) + 2$$

$$T(n/4) = T(n/8) + 1 \implies T(n) = T(n/8) + 3$$

$$\text{After } k \text{ such divisions: } T(n) = T(n/2^k) + k$$

The recursion reaches the base case when $n/2^k = 1$, i.e., $k = \log_2 n$

$$\text{Substituting back: } T(n) = T(1) + \log_2 n = c + \log_2 n$$

Result: $T(n) = \Theta(\log n)$

Interpretation & Scalability

$\Theta(\log n)$ is a very efficient growth rate. As the amount of indexed data (n) grows, the number of operations grows only logarithmically — meaning even a massive increase in data size results in only a small increase in processing steps. This makes the algorithm highly scalable and well-suited for large, continuously expanding search indexes.

Question 9: Data Backup Optimization System

Application

A backup system uses the recurrence $T(n) = 2T(n/2) + \sqrt{n}$ to optimize data-backup operations. The data is divided into two equal halves for recursive processing, with $\sqrt{n}$ work performed at each level (e.g., checksum sampling or partial verification of the backup chunk).

Algorithm Used

Master Theorem for Divide-and-Conquer Recurrences

Algorithm Analysis (Step-by-Step Solution)

Recurrence	$T(n) = 2T(n/2) + \sqrt{n}$
a (subproblems)	2
b (division factor)	2
f(n)	$\sqrt{n} = n^{0.5}$
$n^{(\log_b a)}$	$n^{(\log_2 2)} = n^1$

Comparing $f(n)$ with $n^{(\log_b a)}$: $f(n) = n^{0.5}$ grows strictly slower than $n^{(\log_b a)} = n^1$.

Specifically, $f(n) = O(n^{(1-\epsilon)})$ for $\epsilon = 0.5$, which satisfies Case 1 of the Master Theorem.

Result: $T(n) = \Theta(n)$

Interpretation & Scalability

The $\Theta(n)$ complexity indicates linear growth — the total backup-processing time scales directly and proportionally with the size of the dataset (n). Since the recursive (subproblem) work dominates over the per-level combination cost ($\sqrt{n}$), the algorithm scales efficiently and predictably even as backup datasets grow very large.